A Complete Teacher Failure and Offline Repair Episode

GLEAM bridge example from the procurement reasoning environment

1 Purpose and evidence boundary

This document reconstructs one complete reasoning episode for the following benchmark item.

`L2::bridge_join_0583#L2a`

It connects the benchmark question, the teacher call, the first deterministic execution, the offline rejection, the repair messages, the accepted program, and its deterministic replay.

Three evidence statuses are kept separate. Stored artifacts are copied from the historical JSONL corpus. Reconstructed prompts are regenerated from the committed prompt builder and the stored case fields because the compact corpus did not retain every provider request and response byte for this item. Replay observations are recomputed with the stored accepted plan on the current versioned graph. No reconstructed text is represented as an original provider log.

2 Question and evaluation target

> Just looking at the relevant procurement records, how many contract notices were published by buyers that went on to award a contract to GLEAM SERVICE CENTRE LTD?

The required answer type is a count. The benchmark generator executes the latent bridge specification against its graph snapshot and stores 1,623 as the evaluation oracle. This value is available to the offline evaluator but is not included in a planner or repair prompt.

The intended computation is

```
awards to GLEAM
→ distinct buyers on those awards
→ all procurement records published by those buyers
→ count.
```

3 Teacher Stage 1

The fixed Stage 1 system message is

> You are the typed intent programmer for a UK public-procurement KGQA system. Convert the question into a small computation program. Do not answer the question. Use only information explicitly present in the question. The output shape is enforced by a strict JSON schema.

The case specific user object contains the question, allowed procurement slots, operation guidance, hard rules, and retrieved schema context. Its bridge guidance states that a bridge must use source records, a distinct entity set, a bound target record set, and a final aggregate or selection. The exact object is generated by `question_intent_program_messages` in `src/procurement_graph/reasoning/typed_planning.py`.

The stored Stage 1 response declares a numeric count but proposes only one record filtering step:

```
answer_signature:
  operation: count
  value_type: number
  answer_field_text: ""

program:
  A = filter_records(
        supplier is awarded contract to = GLEAM SERVICE CENTRE LTD,
        buyer publishes contract notice)
  returns: record_set

answer_step: A
```

The response also says that the schema context does not expose a distinct contract notice type or field. It preserves the supplier literal, but it does not represent the central relational requirement. GLEAM first determines a set of buyers, and those buyers determine the population to count.

4 Deterministic compilation and the rejected program

The stored Stage 1 object satisfies the typed intent program contract. It therefore enters the deterministic intent compiler rather than requiring another language model to construct the initial graph. Recompiling the stored object with the committed compiler yields a graph exactly equal, field for field, to the historical rejected graph:

```
variables:
  A:
    kind: record_set
    role: contract_records
    filters:
      supplier = GLEAM SERVICE CENTRE LTD
    depends_on: []

relations: []

return:
  operation: count
  input: A
```

The serialized object has no buyer entity variable and no relation. Its stale template field says `abstain`, while its operative return field says `count`. The runtime uses the compiled return operation and therefore executes the proposal. The historical compact trace does not retain a route tag, but the exact compiler equality means that no initial Stage 2 language model call is needed or attributed to this item. The configured Stage 2 prompt is relevant only when the intent program fast path cannot produce a valid graph.

5 First execution and layered verdict

The executor resolves GLEAM SERVICE CENTRE LTD to a canonical supplier and exhaustively retrieves 35 matching awards. The program returns 35. Grounding, schema validity, operation support, execution, nonempty evidence, and output shape all pass. The online runtime therefore has no basis for initiating a repair.

The offline teacher controller then compares 35 with the hidden oracle 1,623. It records `verified=true`, `oracle_match=false`, and `answer=35`, then exports the plan as a hard negative. The error is semantic rather than syntactic. The program correctly executes the wrong population.

Stage	Observation	Evidence status
Question	Hidden oracle 1,623	Stored benchmark row
Initial proposal	Count supplier records	Stored teacher trace
Initial execution	35; online checks pass	Stored teacher trace
External audit	35 differs from 1,623	Stored trace and hard negative
Repair input	Validation failed; 35 submitted	Audited prompt reconstruction
Repaired execution	$35 \rightarrow 2 \rightarrow 1{,}623$	Stored target and deterministic replay

Table 1: The complete observation sequence.

6 Oracle hidden repair

Oracle agreement is used only by the offline controller to decide whether a repair attempt should be collected. The fixed graph repair system message is

> You are the repair reflector inside a verifier-guided KGQA graph planner. Diagnose the failed graph plan from the structured failure feedback, choose one allowed repair action, and return a repaired understanding network plus repaired graph plan using the SAME schema as the planning step. Never answer the question and never use hidden reference answers. Return strict JSON.

For this item, the visible feedback is equivalent to

```
failure_stage: verifier
failure_reason: wrong_answer
submitted_answer: 35
grounding_issues: []
schema_errors: []
failed_checks: []
allowed_repair_actions:
  - fix_question_type
  - repair_constraints
  - swap_buyer_supplier
  - change_operator
  - change_operation
  - abstain
notes:
  - The submitted answer did not match the hidden verifier.
  - The reference/oracle answer is intentionally hidden from the reflector.
  - Repair the plan using only the question and trace summary.
```

The full repair object also contains the failed graph and online check summaries. Before prompt construction, fields named `oracle_answer`, `reference_answer`, `gold_answer`, and `expected_answer`, as well as keys containing `oracle`, are removed. The compact historical export records the feedback as `submitted answer failed external validation`. It does not retain the raw provider repair completion byte for byte.

7 Accepted repair and deterministic replay

The accepted repair inserts the missing bridge:

```
a1 = record_set(supplier = GLEAM SERVICE CENTRE LTD)
b1 = distinct buyer(a1)
c1 = record_set(buyer in b1)
return count(c1)
```

The stored graph contains two explicit relations. The first binds the buyer field from `a1` into entity set

b1. The second binds those buyers into target record set c1. Deterministic replay exposes the following intermediate populations:

Variable	Kind	Size	Interpretation
a1	Record set	35	Awards whose supplier is GLEAM
b1	Entity set	2	Distinct buyers derived from those awards
c1	Record set	1,623	All records published by the two buyers

Table 2: Intermediate populations in the repaired execution.

The two buyer identities are London Borough of Haringey, Passenger Transport Services, and Royal Borough of Greenwich, managed by GS Plus Ltd. They contribute 1,370 and 253 records respectively. Their sum is 1,623. A representative source record is contract:ocds-h6vhtk-035eee:022611-2022-1, titled "Regular Services" and carrying CPV 60170000. The replay trace retains the identifiers and provenance for every intermediate record; printing all 1,623 target rows is unnecessary for interpreting the transition.

The repaired execution passes the same online checks as the rejected execution and additionally agrees with the hidden oracle. The controller exports the repaired plan as supervised repair data and exports the repaired and rejected plans as an oracle curated preference pair.

8 What changed

The repair did not improve entity grounding, fix invalid syntax, or turn a failing program into an executable one. Those properties already held. It changed the reasoning action by inserting the omitted supplier to buyer to record bridge. The result changed from the number of GLEAM awards to the number of records published by buyers that had awarded GLEAM a contract.

This distinction is the purpose of the environment. It exposes the proposed program, every intermediate population, the final result, and the authority of each verdict. The environment does not itself prove that an agent has improved. It makes the change and its consequences observable enough to attribute a successful repair.

9 Artifact locations

All historical objects are joined by L2::bridge_join_0583#L2a.

Artifact	Role
data/qa/cicada_core_v4/train.jsonl	Question and hidden oracle
data/qa/teacher_full_v1/traces.jsonl	Briefing, rejected graph, answer, verdict
data/qa/teacher_full_v1/hard_negatives.jsonl	Offline rejected example
data/qa/teacher_full_v1/repair_sft.jsonl	Visible failure signal and accepted target
data/qa/teacher_full_v1/dpo_pairs.jsonl	Accepted and rejected preference pair
src/procurement_graph/reasoning/typed_planning.py	Prompt and repair message builders
scripts/eval_targeted_v2.py	Hidden mismatch feedback construction
scripts/run_teacher.py	Offline routing and export conditions